

15 – RTOS scheduling

Conditions for possible scheduling of tasks on a RTOS

Jérôme Landré – jerome.landre@ju.se

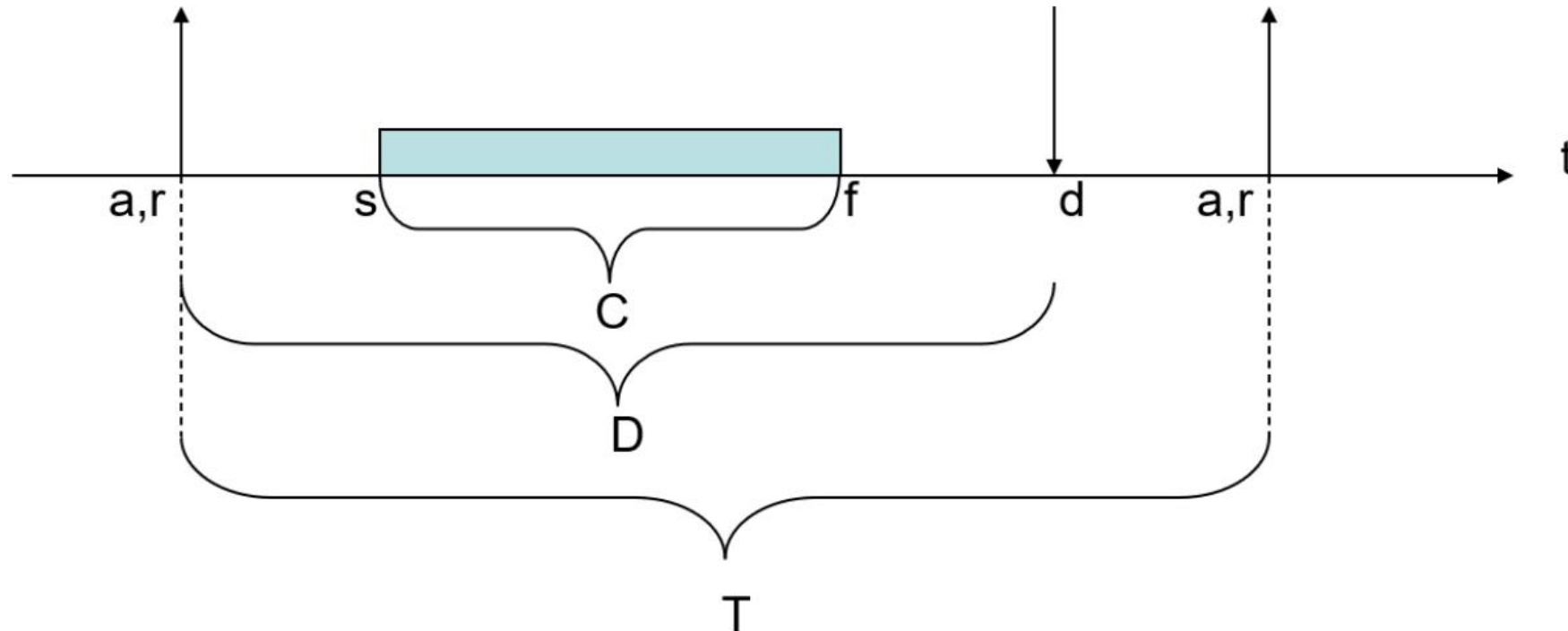
OUTLINE

- 1) RTOS scheduling
- 2) Tasks timing properties
- 3) CPU Utilization
- 4) Rate-Monotonic Scheduling (RMS)
- 5) FreeRTOS scheduling

1) RTOS SCHEDULING

- In a **RTOS**:
 - a) Is there a way to organize tasks to ensure Real-Time constraints?
 - b) Is there a way to compute the feasibility of a list of tasks?
- a) Yes: sort tasks as periodic and aperiodic (non-periodic)
 - Periodic task will be scheduled with the rate-monotonic scheduling (RMS) algorithm
 - Aperiodic tasks will be scheduled with the Round-Robin (or another algorithm)
- B) Yes: using the CPU utilization factor and “Liu & Layland” bounds!

2) TASKS TIMING PROPERTIES



- a Arrival time, when a task becomes ready for execution
- r Release time or request time; it is also referred as arrival time
- s Start time, time when task starts its execution.
- f Finishing time, the time at which a task finishing its execution.
- d Deadline, is the time before which a task should be complete.

3) CPU UTILIZATION

- For one process P, we define the CPU utilization U as:

$$U = \frac{C}{T}$$

where C=computing time, T=period

- For a set of Processes P_i , we define the CPU utilization U as:

$$U = \sum_i \frac{C_i}{T_i}$$

3) CPU UTILIZATION

- To be possible to schedule, a set of task should have an utilization of the CPU less than 1:

$$U = \sum_i \frac{C_i}{T_i} \leq 1$$

3) CPU UTILIZATION

- Examples of a CPU utilization computing...

4) RATE-MONOTONIC SCHEDULING (RMS)

- Each process is assigned a (unique) priority based on its period; the shorter the period, the higher the priority
- For two processes i and j :

$$T_i < T_j \Rightarrow P_i > P_j$$

- It means the scheduler will run with a high priority the tasks that have a lower period of time

4) RATE-MONOTONIC SCHEDULING (RMS)

In [computer science](#), **rate-monotonic scheduling (RMS)**^[1] is a priority assignment algorithm used in [real-time operating systems](#) (RTOS) with a static-priority scheduling class.^[2] The static priorities are assigned according to the cycle duration of the job, so a shorter cycle duration results in a higher job priority.

These operating systems are generally [preemptive](#) and have deterministic guarantees with regard to response times. Rate monotonic analysis is used in conjunction with those systems to provide scheduling guarantees for a particular application.

4) RATE-MONOTONIC SCHEDULING (RMS)

Least upper bound [\[edit\]](#)

Liu & Layland (1973) proved that for a set of n periodic tasks with unique periods, a feasible schedule that will always meet deadlines exists if the CPU utilization is below a specific bound (depending on the number of tasks). The schedulability test for RMS is:

$$U = \sum_{i=1}^n U_i = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

where U is the utilization factor, C_i is the computation time for process i , T_i is the release period (with deadline one period later) for process i , and n is the number of processes to be scheduled. For example, $U \leq 0.8284$ for two processes. When the number of processes tends towards infinity, this expression will tend towards:

$$\lim_{n \rightarrow \infty} n(\sqrt[n]{2} - 1) = \ln 2 \approx 0.693147 \dots$$

Therefore, a rough estimate when $n \geq 10$ is that RMS can meet all of the deadlines if total CPU utilization, U , is less than 70%. The other 30% of the CPU can be dedicated to lower-priority, non-real-time tasks. For smaller values of n or in cases where U is close to this estimate, the calculated utilization bound should be used.

In practice, for the i^{th} process, C_i should represent the worst-case (i.e. longest) computation time and T_i should represent the worst-case deadline (i.e. shortest period) in which all processing must occur.

4) RATE-MONOTONIC SCHEDULING (RMS)

- The bound depends on the number of tasks:

N	Utilization bound (limit)
1	
2	
3	
5	
10	
100	
∞	

$$U = \sum_{i=1}^N \frac{C_i}{T_i} \leq N(2^{\frac{1}{N}} - 1)$$

4) RATE-MONOTONIC SCHEDULING (RMS)

- There are two conditions to ensure the possible schedule:

- Necessary but not sufficient:

$$U = \sum_i \frac{C_i}{T_i} \leq 1$$

- Sufficient:

$$U = \sum_{i=1}^N \frac{C_i}{T_i} \leq N(2^{\frac{1}{N}} - 1)$$

3) CPU UTILIZATION

- Examples of CPU bound utilization computing...

4) RATE-MONOTONIC SCHEDULING (RMS)

- Another way of computing the scheduling of a set of tasks is to use hyperbolic bound scheduling condition:
 - A set of tasks T_i , with $i=\{1,\dots,N\}$ is schedulable when:

$$\prod_{i=1}^N (U_i + 1) \leq 2$$

4) RATE-MONOTONIC SCHEDULING (RMS)

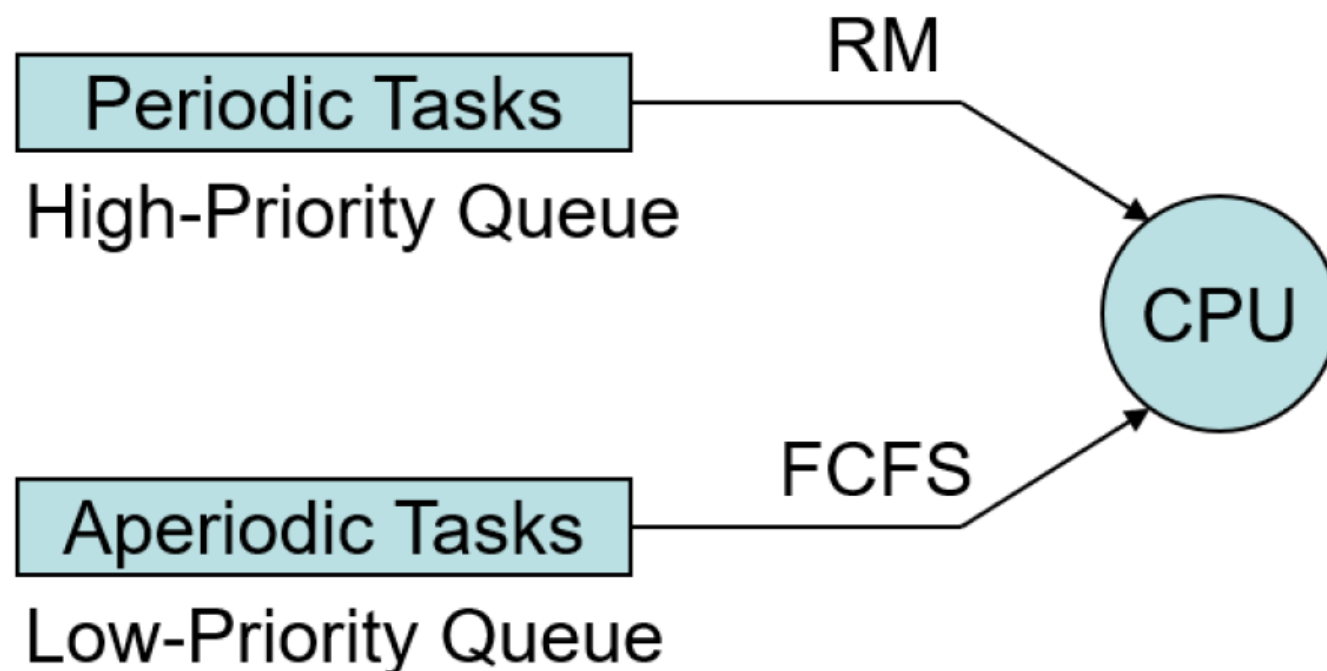
- Examples of hyperbolic bound condition...

5) FREERTOS SCHEDULING

- In FreeRTOS, RMS is used for periodic (high-priority) tasks (to ensure real-time feasibility) and another algorithm can be used for aperiodic (low-priority) tasks

Around 70% of CPU time

Around 30% of CPU time



RM(S): Rate Monotonic (Scheduling)

FCFS: First Come First Serve (FIFO)
Can also be Round-Robin or another
Scheduling Algorithm

ANY QUESTIONS?